

CALIFORNIA STATE UNIVERSITY, NORTHRIDGE

Real Time Facial Recognition and Tracking System Using Drones

A thesis submitted in partial fulfillment of the requirements
For the Degree of Master of Science in Computer Science

By
Artak Melkumyan

December 2021

Copyright by Artak Melkumyan 2021

The thesis of Artak Melkumyan is approved:

Ani Nahapetian, PhD

Date

Taehyung Wang, PhD

Date

Katya Mkrtchyan, PhD, Chair

Date

Table of Contents

Copyright	ii
Signature Page	iii
Abstract.....	v
1. Introduction	1
2. Related Work.....	4
3. Methods.....	11
3.1 Drone Operation.....	12
3.2 Facial Recognition Systems.....	14
3.3 LBPH Implementation	16
3.4 FaceNet Implementation.....	18
3.5 Face_Recognition Implementation.....	20
4. Experiments	23
4.1 Experiment Design	23
5 Results.....	29
5.1 System Performance.....	29
5.2 Facial Recognition at Angles	31
5.3 Facial Recognition from A Distance	35
5.4 Facial Obstruction	39
5.5 Target Recognition and Tracking.....	43
6 Conclusion	46
References	47

List of Figures

Figure 3.1 System Overview Flowchart	11
Figure 3.2 Ryze Tech Tello Drone	12
Figure 3.3 Drone System Overview	14
Figure 3.4 Basic Facial Recognition Process	14
Figure 3.5 LBPH Implementation Flowchart	18
Figure 3.6 FaceNet Implementation Flowchart	19
Figure 3.7 Face_Recognition Implementation Flowchart	23
Figure 4.1 LBPH Output with FPS Tracker	24
Figure 4.2 Angle Test Degree Demonstration	25
Figure 4.3 Facial Obstruction Items Used	27
Figure 5.1 FPS Test Results	30
Figure 5.2 Angle Test Results – One Individual	32
Figure 5.3 Angle Test Results – Two Individuals	33
Figure 5.4 Angle Test Results – Three Individuals	34
Figure 5.5 1 Meter Distance Test	36
Figure 5.6 1.5 Meter Distance Test	37
Figure 5.7 2 Meter Distance Test	38
Figure 5.8 2.5 Meter Distance Test	39
Figure 5.9 Glasses and Sunglasses Obstruction	40
Figure 5.10 Mast Obstruction	42
Figure 5.11 Vertical Obstruction	43

Abstract

Real Time Facial Recognition and Tracking System Using Drones

by

Artak Melkumyan

Master of Science in Computer Science

The goal of this thesis is to create a system with a drone capable of autonomous facial detection and recognition, used to find and track a specified individual in real time. The automated system works by having a drone be responsible for video capture and streaming, while a computer, receiving that video stream, will run modern day facial recognition algorithms and relay instructions back to the drone. These instructions will control the drone's movements with the goal of identifying and tracking a singular specified individual. Three facial recognition systems were implemented and tested to see which works best in this system. The three facial recognition systems being tested are Local Binary Pattern Histogram (LBPH), FaceNet, and Face_Recognition, all of which were chosen due to a mixture of their performance and prominence. Experiments were conducted to quantify the performance of each facial

recognition system with respect to the distance of the drone from an individual, while also taking into consideration the angle of an individual's face and common accessories that cause facial obstruction. Additional experiments were conducted to measure each facial recognition system's performance by examining how many frames per second (FPS) each system could analyze. The results showed that the implemented automated drone system that uses facial recognition technology to track an individual in real time is practical. The algorithms used in this thesis reported high accuracy rates in multiple tests; however, there are performance limitations that require further work and optimization.

1. Introduction

Facial recognition technology has come a long way and it is still rapidly advancing. To understand the facial recognition systems used in this thesis, we must look towards Convolutional Neural Networks (CNN). CNNs are nothing new, having been around since the 1980s. They are a class of deep learning algorithms which can take in an image as input and assign importance, learnable weights, and biases to various aspects or objects in the image. These assigned characteristics can then be used to differentiate one object from another. A CNN can identify an object in an image with the use of convolutional layers. These layers contain filters which can detect rudimentary patterns, like an edge or a line. As more convolutional layers are added, more precise filters can be put into place. Cycling through several iterations of the filtering process, a CNN can go from finding edges and lines to shapes, and finally objects, people, or anything else it is designed for. As computational power continues to grow, this technology can now be utilized much more quickly and efficiently, like in conjunction with a live video stream. CNNs have become the main method used for facial recognition systems. Facial detection and recognition algorithms can use pre-existing models that have been trained using thousands of images and hundreds of hours of computational time. In this thesis, these pre-trained models will be used in conjunction with the selected facial detection and recognition algorithms to create a drone capable of detecting and identifying an individual, along with tracking that individual's movements.

Even though computational power has grown, most CNNs require a high-end Graphics Processing Unit (GPU) that requires too much power and weights too much to function in low cost, light weight drones. One way to alleviate this issue is to stream the images captured on

the drone to a computer with adequate computational power. Drones communicate with outside devices through 2.4 gigahertz radio waves, which many Wi-Fi capable devices are easily able to connect to. The computer that is connected to the drone can utilize this network to communicate back and forth with the drone. Having a much more powerful system handle image analysis will result in faster processing time and greatly improve the drone's reaction times for tracking purposes. However, the network does come with drawbacks, there is a chance that communication between the drone and computer can be lost in short bursts or even disconnected entirely.

The network established between the computer and the drone is capable of two-way communication. The drone can stream a live video feed to the computer, where analysis is performed, and the computer can send instructions to the drone that dictate its movement, all in real time. The drone can also send status updates, like the battery percentage, flight time, and current height to the computer. This network is key for the system to function, as the drone will be oblivious without input from the computer. Due to the system failure caused by disconnection, maximum range of a drone's network is an important consideration.

The final piece of the system implemented in this thesis is the facial tracking component. A single individual can be indicated as the target in the program and when that person is found, with a minimum 80% confidence level, a special flag will be set notifying the tracking portion of the system. Tracking works by calculating the area of the target individual's face in the frame they were identified in. A small area indicates that the target is far away, and a large area indicates that they are too close. The computer will handle this analysis and send movement instructions to the drone so that the target is kept in a specified range relative to the drone.

The center of the individual's face is also identified because the target can turn away from the drone. If the target turns away from the drone, the computer will find and relay the best movement instructions that cause the drone to directly face the target; as testing showed, this is the best position for facial detection and recognition. The tracking algorithm only sends instructions for one frame at a time, so it is important for the system to quickly analyze frames. If one of the facial detection or recognition systems is too slow, then it might run at a delay, causing it to examine frames from seconds ago, leading to incorrect instructions being relayed to the drone.

2. Related Work

Recently, drone has turned somewhat into a buzzword as the excitement around drones is ever growing and the costs associated seem to be decreasing. Due to this ease of access, many people have been able to research and implement computer vision related projects while utilizing drones. The main points of this thesis center around implementing a system that performs real time facial recognition and tracking using a drone. Though some researchers have done similar work, they did not propose the same real time automated facial recognition system that uses drones to track individuals, as this thesis does.

Researchers at the Institute of Information Science, Academia Sinica were able to create a facial recognition system that uses drones. They implemented two facial recognition systems, Face++ and ReKognition, then used them to conduct empirical tests to evaluate the efficacy of drone facial recognition considering factors that may affect performance such as distance from a subject, angle of depression, and drone height. Distance, angle of depression, and height all refer to the drone position as shown in Figure 3.3.

Photos were taken as the drone rose to heights of 3, 4, and 5 meters. The drone started at 2 meters away from the subject and photos were taken every .5 meters as the drone slowly backed up to 17 meters away from the subject. The researchers found that as drone height and distance from the subject increased, the accuracy drastically decreased. The accuracy for Face++ and ReKognition were both 100% at 1.5m for the drone height and 2m for the drone distance from the subject. However, as distance from the subject and drone height increased the accuracy of both facial recognition systems dropped drastically. From 17 meters away

Face++ averaged .5% accuracy from 2-5 meters of drone height. ReKognition averaged 5% accuracy for the same distance, a large improvement but still greatly lacking in accuracy [5]. Other issues of note were situations where the distance between the drone and subject was small, but the drone's height was multiple meters in the air. Situations like this resulted in a high angle of depression, which led to worse accuracy and an increase in false positives, where a person was identified as someone else. This paper showed that implementing facial recognition systems in conjunction with a drone is plausible; however, the results showed that there are limitations of such a system are numerous and can have detrimental effects on accuracy.

At the Institute of Information Science and Technologies of the National Research Council (ISTI-CNR) in Pisa, researchers also set out to study the restraints holding back facial recognition use in conjunction with drones. The main constraint they considered was that with a drone at a high altitude, people in the frame could appear in a low resolution. This would hinder facial detection and recognition systems as a low quality image makes it harder to differentiate things in the frame, so they looked for ways to improve on it. They implemented three models ResNet-50, SENet, and their own model for facial recognition. For facial detection they used dlib, MTCNN, and OpenCV-DNN. All three systems were trained using the same dataset and tested using the DroneSURF dataset, which contains both low and high quality images. Testing consisted of taking a low quality image, running a facial detection system on it, then using a facial recognition system to find a match with its high resolution counterpart. Initially, the accuracy was very low, the best performing system, SENet, only had an accuracy of 24.25% using MTCNN. They hypothesized that the low accuracy was caused by the cropping of the

detected faces and tried the tests again with the faces having a tighter cropping which left out more of the background. The results improved dramatically with the highest performing system being the model they designed with an accuracy of 60.87% using MTCNN. SENet's performance also drastically improved, now reporting an accuracy of 53.56%. This paper showed that when it comes to low quality images, the cropping of the detected face is very important and that a tighter cropping may result in better accuracy [1].

Other researchers at the Xi'an Aeronautical University used an LBPH face recognizer to create a security system using drones. The researchers used a Raspberry Pi attached to their drone to run the facial recognition system. The system used Haar Cascade to detect faces in a crowd, then had those faces cropped and scanned by a trained LBPH facial recognizer. If the facial recognizer found a match in its dataset it would notify the user so they could take over manual control of the drone or assign the drone new instructions. Training the facial recognizer consisted of using 250 images of each person and cropping them to only include the face found in the image. The cropped image was then resized and converted to black and white in order to decrease the amount of information in the photo, speeding up the time required to train the model. The system had a minimum confidence level of 50% for positively identifying a person. This minimum confidence level was used because there were many issues, especially distance related ones, that came up throughout testing. Often, a person was detected but the recognition system wasn't confident enough to label them. The confidence requirement was slowly decreased to find a point that maximized recognition but also limited false positives. Overall, this system was able to identify individuals in a crowd with an accuracy of 89.1% [11].

Researchers at The LNM Institute of Information Technology also looked towards LBPH for their facial recognition system. They implemented two facial recognition systems, LBPH and FaceNet. Like many other researchers they implemented these systems on a Raspberry Pi and attached it to a drone. Having a Raspberry Pi on a drone provides many advantages because all necessary computation can be done on it; however, it isn't a notably powerful device. The drone was able to use the two facial recognition systems to find a specific target in a crowd autonomously. Despite several issues, the system did function; though they concluded that creating an accurate model that could find a face from a crowd is a very challenging task due to hardware limitations of the Raspberry Pi [7]. The system implemented in this thesis will solve that issue by streaming the drone's video to a computer with the necessary computational power to accurately examine each frame and relay information back to the drone.

Researchers at the RMK College of Engineering and Technology in Chennai conducted a very similar experiment. In their system they also used Haar Cascade to detect faces and LBPH for facial recognition. However, they reported a much better accuracy rate of 98.6% [6]. A possible factor for this high accuracy rate could be that their system would compare the captured faces against every single test image available. The drawback of this approach was that it made the system extremely slow, only being able to process about 9 FPS. The LBPH system used by the researchers at the RMK College of Engineering and Technology and the researchers at Xi'an Aeronautical University, were implemented in much the same way. This thesis also implemented LBPH and used Haar Cascade for the facial detection portion of the system. However, this thesis differs in that when a specified face is found, the drone will autonomously

begin tracking the individual instead of relaying that information back to the user and waiting for input; like the system designed by the researchers at Xi'an Aeronautical University.

Stanford University researchers developed a system known as Deep Drone for identifying and tracking objects. In their paper they mainly focused on identifying and tracking people, but not a specific person, as the system in this thesis does. Deep drone is comprised of two main systems; the first is a detection algorithm, running a CNN designed to find any people in the frame. The second system is the tracking algorithm which uses histogram of oriented gradient (HOG) features and a kernel correlation filter (KCF). Detection of a person was extremely slow and computationally heavy, allowing the system to only scan 1.6 FPS. Once a person was detected the system would draw a box around them and pass on the location to the tracking algorithm. Since KCF was only performing tracking on the frames, it was able to process a lot faster at 71 fps [4]. One drawback of this system is that if the subject moves too quickly, or something else happens and the tracking algorithm loses the target, then the system will have to go back to using the detection algorithm as the tracking algorithm is unable to locate people. Some challenges to their system included side profiles and targets that were constantly moving around; they solved this by lowering the confidence level required to flag an object as a person. Lowering the confidence level seems like a quick fix but a low confidence level can result in more false positive matches. Another issue was that state-of-the-art systems were running too slowly on the drone, and it couldn't issue movement commands quickly enough to track the target. That is why they ended up using the KCF tracking system because it required less computational power; however, this older system did result in a loss of accuracy. The system implemented in this thesis is different from Deep Drone as Deep Drone is designed to find and

track any person that it detects. However, the system implemented in this paper will have a singular target individual identified in the program, and it will autonomously find and track only that individual.

Researchers from the North Carolina State University, Raleigh have been working to solve many of the aforementioned problems faced by researchers. Mainly, they tackled the issues faced by the researchers at the Institute of Information Science, Academia Sinica, where they found that the height of drone played a huge factor in facial recognition. For their work, the researchers used 3 models VGG16, VGG19, and InceptionResNetV2. They selected these models as they are some of the most accurate models currently in use. They trained these models using the DroneFace dataset which was designed to help advance current drone capabilities. These images were taken at different heights, 1.5 to 5 meters high and at different distances, 2 to 17 meters from the subject. The 3 models were all then trained based on these images but were trained differently for each test. The tests were designed to find the model that could best deal with a change in the drone's altitude when it came to facial recognition. The models were trained using the images taken from 0 – 4 meters in drone height but tested on images taken at 5 meters of drone height and vice versa. The researchers found that models were more accurate when the height of the drone was higher during training than it was during the tests. However, they found that InceptionResNetV2 and VGG16 showed that they had the capability to identify faces at a height higher than that of the height they were trained on. They also found that identification tasks were affected much less when the height differed compared to the verification tasks. All 3 systems kept an accuracy of over 90% during identification tasks but only had an accuracy of around 34-38% during validation tests when trained on a height lower

than the height of the test images [2]. Concluding their research, they found that training the model with images that are further away and with a higher drone height helped improve the accuracy of their systems. They also found that the model with the most consistent and accurate results was InceptionResNetV2.

3. Methods

In the methods section, the frameworks involved in implementing the system created in this thesis will be explored. The system will function by having a drone stream the video feed from its camera to a computer. That computer will run facial detection and recognition algorithms to perform real time analysis on each frame of the video stream in search of a singular specified individual. If that individual is found, the tracking portion of the system will activate, and the computer will send movement instruction to the drone, designed to keep the specified individual in frame. All these smaller systems working together will result in an autonomous drone capable of finding and tracking a specified individual.

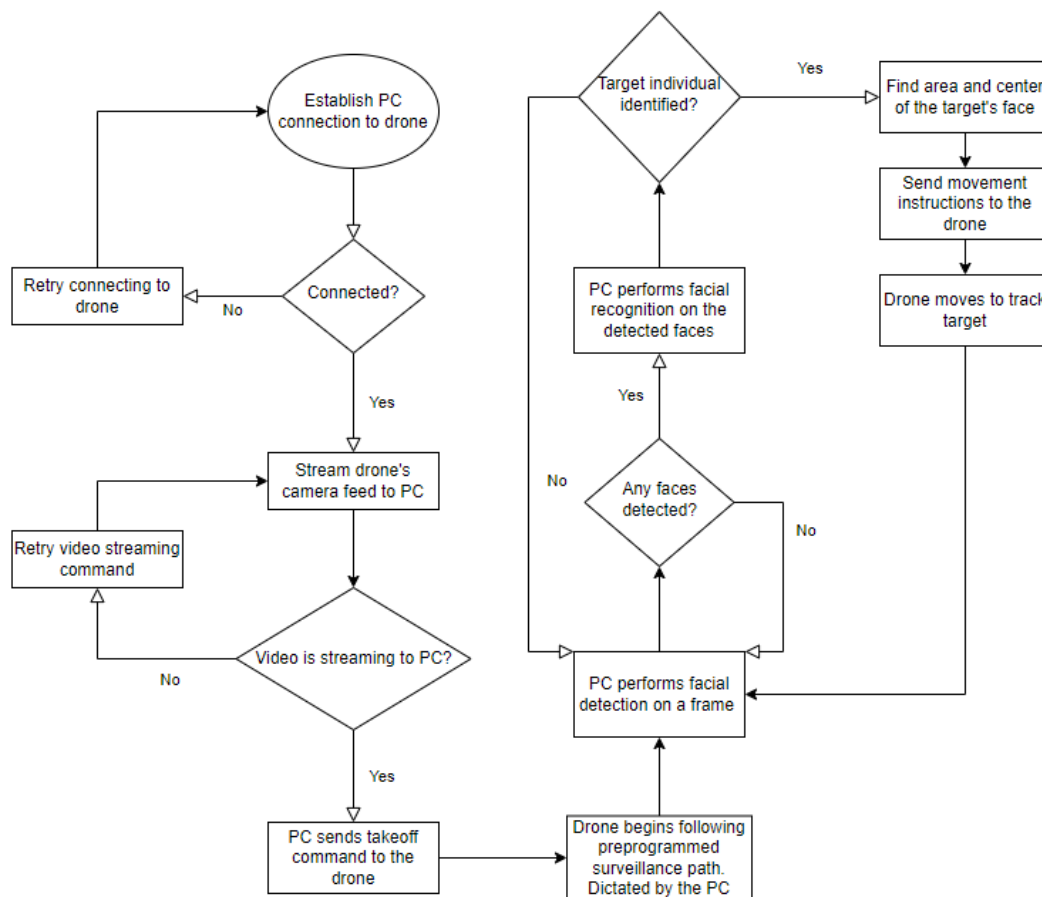


Figure 3.1 A flowchart depicting the general overview of the steps taken by the system implemented in this thesis.

3.1 Drone Operation

The use of drones has become more common place in recent years as they became more widely available. Nowadays, people are using drone technology in ways that were previously unimaginable; like delivering packages, planting trees, and even quickly deploying medical equipment to those in need. A drone is comprised of 11 basic components consisting of: the frame, motors, propellers, electric motor controller (ESC), power distribution board (PDB), flight controller, battery, receiver, camera, video transmitter (VTX), and sensors. Drones also have four degrees of freedom. Those four degrees are made up of three translations and one rotation. The three translations allow drones to move up and down, left and right, and forwards and backwards. The one rotation allows drones to rotate clockwise and counterclockwise. When working with the drone, parameters like height, distance, and angle of depression are kept in mind. All of these parameters are shown in relation to the drone in Figure 3.3

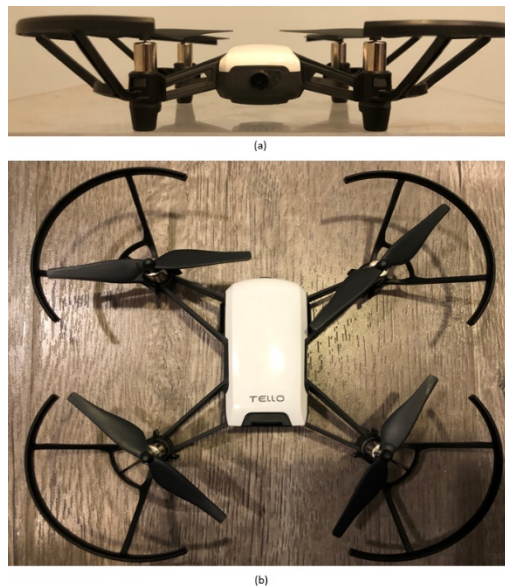


Figure 3.2 A Ryze Tech Tello Drone with a length of 9.9cm, width of 9.1cm, and a height of 4.1cm. (a) is a view from the front and (b) shows a top-down view.

For this thesis, a Ryze Tech Tello drone was used, as shown in Figure 3.2. It is a small lightweight drone weighing only 80 grams but boasting a 13 minute flight time, 100 meter flight distance, and a 5 megapixel camera which is able to transmit HD video in 720P. The djitellopy python library was also utilized for this project making it easy to send commands and receive information from the drone directly to a computer. Using the connect() function, a connection could be established between the drone and a computer. The streamon() function then transmits the video captured from the drone directly to a computer allowing the image to be examined for facial recognition purposes. Even though a live video is being streamed, it is broken down frame by frame for examination by the facial recognition system. The system is also able to display the stream on the computer and provide additional information if people are identified.

The djitellopy library also contained many different functions designed for controlling drone movement, these functions were used to implement the tracking portion of this project. The user can specify a person as a target in the system and when a specified target is found, the drone will track them. As images are examined and people are specified, a box is drawn around each face, displaying who the system has identified them as. If one of those identified people are the target, the system uses the size of the box drawn around them to issue movement commands. If the area of the box is decreasing, or already small, the drone will move closer until it reaches a specified range. It works the same for targets if the area is too big or the target is getting closer. Similar movement commands are used to rotate the drone if the target moves side to side, insuring they can be tracked. However, all tracking optimally should be done with

the target's frontal face being visible. Side profile images lead to reduced accuracy and images of the back of the head lead to no accuracy at all, as discussed in the results section.

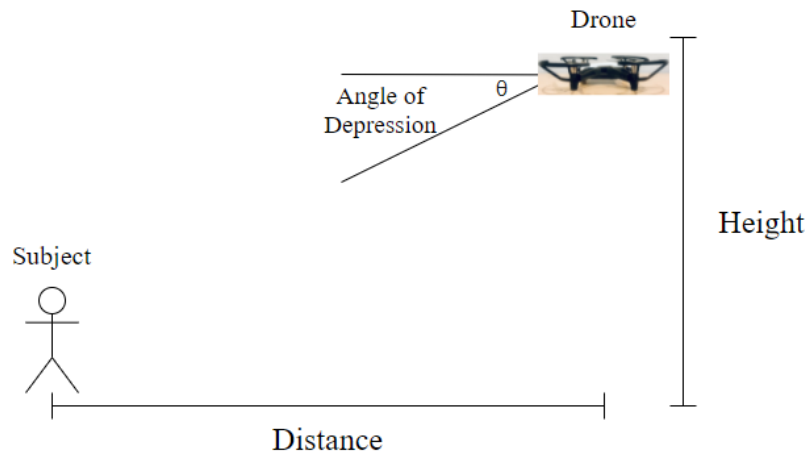


Figure 3.3 The distance from the subject to the drone refers only to the horizontal separation of the two. Height refers to the drone's height relative to the ground. The angle of depression is the angle of the drone's camera relative to the subject as denoted by θ .

3.2 Facial Recognition Systems

Most modern-day facial recognition systems work as shown in Figure 3.4. Starting with an input image, the system scans it for any faces. If any faces are found, they are cropped and feature extraction is used to create a face embedding, a vector containing information about facial features that can be used to identify someone. That embedding, or the facial features, are then passed on to a classifier. The classifier is a pretrained model that contains information about one or more people that it was trained to classify. Using the incoming data and working with any minimum confidence threshold value set, the classifier is able to return information regarding the person(s) identified. All three systems used in this thesis, work in much the same way.

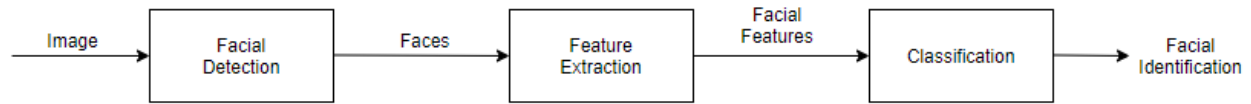


Figure 3.4 Overview of modern facial recognition systems process.

For this project, three different facial detection and recognition systems were implemented to test the capability of the system implemented in this thesis. LBPH face recognizer, FaceNet, and Face_Recognition were the systems of choice. LBPH face recognizer was chosen because previous work had been done utilizing it in conjunction with drones [6;7;11] and it serves as the baseline for results. FaceNet and Face_Recognition were chosen because of their outstanding performance in the Labeled Faces in the Wild (LFW) benchmark dataset. LFW is a public dataset used to measure the accuracy of a system with respect to face verification. Using the dataset FaceNet reported an accuracy of 99.63% and Face_Recognition reported a 99.38% accuracy.

All three systems used a test dataset to train the classifier for specific individuals to identify. The test dataset consisted of 15 labeled images of each individual. Five images were taken in a setting with low light, five images in a setting with ambient lighting, and five images were taken in a setting with high amounts of light. The images were all taken showing a frontal or side view of an individual's face in front of a plain background. Each system also performed some uniform actions on the images provided to it. The test images provided were all converted to black and white and resized for training purposes. The systems also located the face in each photo and cropped it, so only that part of the image was used for training. However, all three of the systems did not produce the same cropping of a face, as they have different importance values assigned to different areas of the face.

Another uniform process occurs when the drone is streaming images to the computer. The images sent by the drone are very large and colorful, so each system resizes the image and converts it to black and white, this helps speed up the facial recognition process. Each system also displays the stream in a window for the user. Every system draws a box around each face detected in the frame to show the user that a facial detection has occurred. If a face matches any in the trained classifier, then a name will be appended to the box; if the face is unknown, then it will say unknown. Whenever a specified target is found the tracking portion of the program will activate.

3.3 LBPH Implementation

LBPH face recognizer was selected due to the amount of previous work utilizing this system [6;7;11]. LBPH works by selecting a pixel and a group of pixels around it in a specified radius. The values gathered are then used to create a histogram, the pixel level histograms are then combined to create a regional level histogram. Finally, the regional level histograms are combined to build a histogram which provides a general description of the detected face [8]. This general histogram can then be used for comparison with a classifier to find similar histograms which would help identify a person. LBPH typically uses 4 main parameters: radius, neighbors, grid X, and grid Y. Radius is used to build the circular local binary pattern which represents the radius around the central pixel and usually the value is set to 1. Neighbors are the number of sample points used to build the circular local binary pattern and it is usually set to 8. Grid X and grid Y each represent the number of cells in the horizontal and vertical plane, respectfully. More cells result in a finer grid with higher dimensionality for the resulting feature vector, this value is usually set to 8.

The first step in implementing the LBPH system is creating the model. To aid in the model's creation Haar Cascade is used, a facial detection algorithm first proposed in 2001 [10]. This system works by combining increasingly more complex classifiers into a cascade. In this project the frontal face cascade was used, which contains classifiers that excel at frontal face detection. To create the trained model LBPH face recognizer is used in conjunction with Haar Cascade. This system starts by taking the provided labeled images and converting them to grayscale, Haar Cascade is then used to detect the faces in each image. The detected faces are then cropped from the image and sent to the LBPH face recognizer which uses them as training images to create a model.

The model is then loaded into the facial recognition system. In this system, the drone constantly streams live video which is broken up frame by frame and examined. Haar Cascade is then used to detect any faces in each frame, any detected faces are stored in an array where the trained LBPH face recognizer algorithm can run and return any matches. An overview of this system is shown in Figure 3.5.

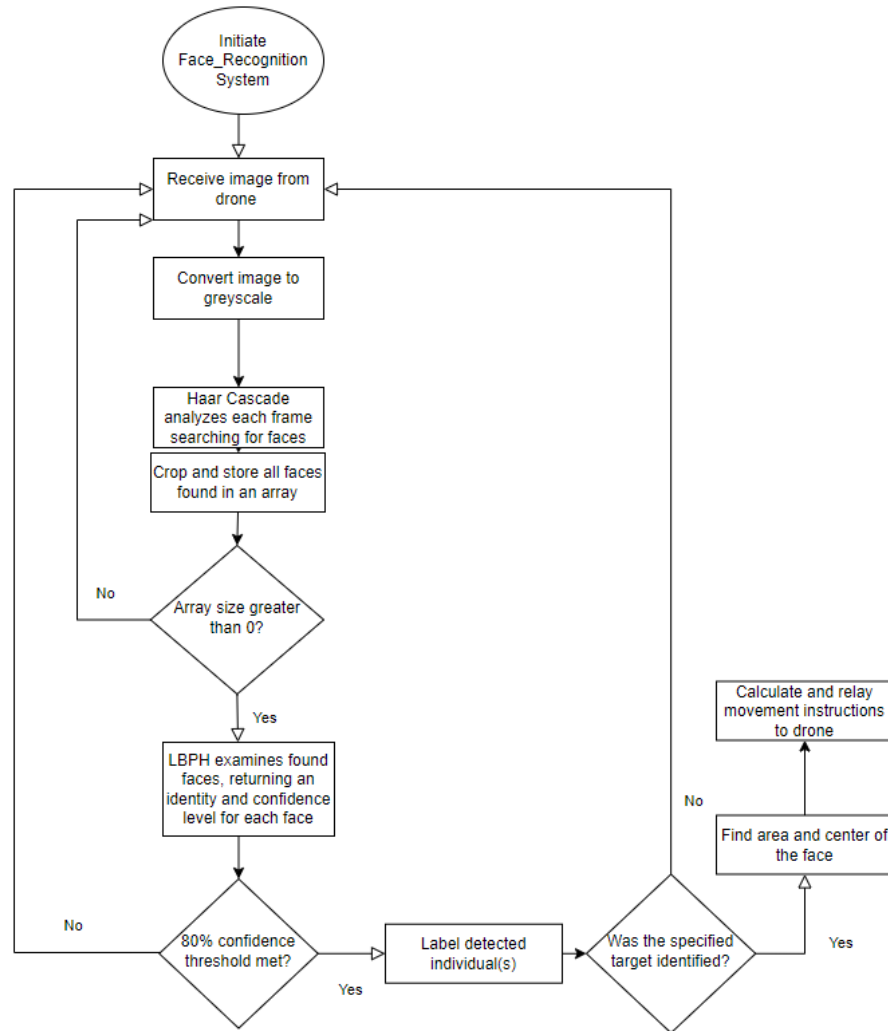


Figure 3.5 Flowchart of the LBPH facial detection and recognition systems.

3.4 FaceNet Implementation

FaceNet is a unified embedding system proposed by Google researchers [9]. It works by creating a Euclidean embedding for an image using a deep convolutional network. Those embeddings contain 128 face measurements that are then mapped to a Euclidean space where images of the same person are clustered together. It is unknown what the network uses to base its measurements off when creating the embedding, but the results speak for themselves.

When an unknown image is presented, the algorithm maps the Euclidean embedding and classifies it as which ever known person is closest to it.

For this thesis, the TensorFlow implementation of FaceNet was used. To implement FaceNet there are three main steps to follow. The first is preprocessing, where all the faces in the training data set are cropped and aligned. The next step involves embedding or learning representations of faces in multi-dimensional spaces where distance corresponds to the measure of face similarity. The training dataset was used for embedding and creating a classifier. Finally, we have the classifier which works in conjunction with the FaceNet model and provided Tensorflow detect_face function.

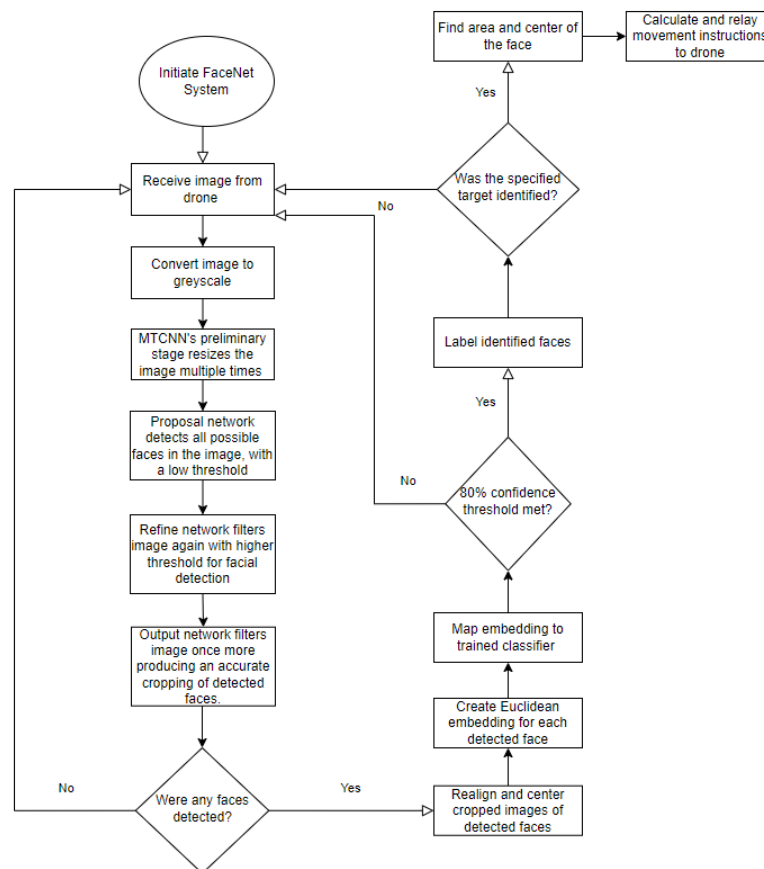


Figure 3.6 Flowchart of the FaceNet facial detection and recognition systems.

Each frame streamed to the program by the drone is resized, converted to black and white, and examined. Detect_face is used to create a Multitask Cascaded Convolutional Neural Network (MTCNN) which is used to find faces in a frame and store them in an array. MTCNN works in three stages, proposal network, refine network, and output network. There is a preliminary process that runs before the first stage. The preliminary process consists of resizing the image multiple times to help detect faces of varying sizes. After the preliminary process is complete the first stage, proposal network, begins. Proposal network is comprised of several smaller process, its first process, performs facial detection with an extremely low threshold. This low threshold for facial detection causes anything in the frame even remotely resembling to a face to be labelled as one. This process results in numerous false positives but makes sure that all possible faces in the image are detected. The resulting areas are considered the “proposal” and sent to the second stage, the refine network. In this stage, the area is filtered, and more precious bounded boxes are created around the detected faces. Lastly, the output of the refine network is used as the input for the final stage, the output network. In this stage the final refinements of the bounded boxes are performed resulting in a very precise output of the detected face. Those images are then mapped to the trained classifier and if the minimum threshold value is met, the person's identity is returned. A flowchart of this process can be found in Figure 3.6.

3.5 Face_Recognition Implementation

Face_Recognition is the self-proclaimed “world’s simplest face recognition library” [3].

Face_Recognition is built using dlib, a toolkit containing machine learning algorithms for complex software, specifically it uses dlib's face recognition algorithms built with deep learning.

Face_Recognition uses histogram of oriented gradient (HOG) to detect faces in an image. HOG works by selecting a pixel in the image and looking at the surrounding pixels with the goal of figuring out how dark the current pixel is compared to the ones around it and in which direction the image gets darker. The program records which direction the image gets darker and after repeating the process for every single pixel we end up with a gradient showing the flow of light across the entire image. The image is then broken up into 16 by 16 squares and the direction is decided for each square based upon the most prevalent direction in that section. That version of the image is then compared to the HOG face pattern, formed from training on many faces, which finally determines whether the image contains a face and where it is. Like FaceNet, Face_Recognition also aligns and centers its images. It uses a face landmark estimation algorithm to map 68 specific points on the face which allows the program to realign or center the image as needed. Face_Recognition uses a deep CNN to generate a 128-measurement embedding of each face from the training data set. This is the same system invented by the Google researchers that created FaceNet.

For the implementation of Face_Recognition, the test data set was loaded, and encodings were generated. An overview of the steps taken by the Face_Recognition system for facial detection and recognition are found in Figure 3.6. Like the other systems, it reads the incoming video stream frame by frame and resizes the frames along with converting them to black and white. HOG was then used to locate faces in the frame and face encoding was used to generate an embedding for each face that was found. Finally, those embeddings were compared with the known face embeddings and the result was returned. The implementation of Face_Recognition was a lot quicker than LBPH or FaceNet, taking only a fraction of the time. This is because of the

Python Face_Recognition library where many of the implementation steps mentioned above are available as functions in the library. In LBPH and FaceNet most of the functions had to be created from scratch.

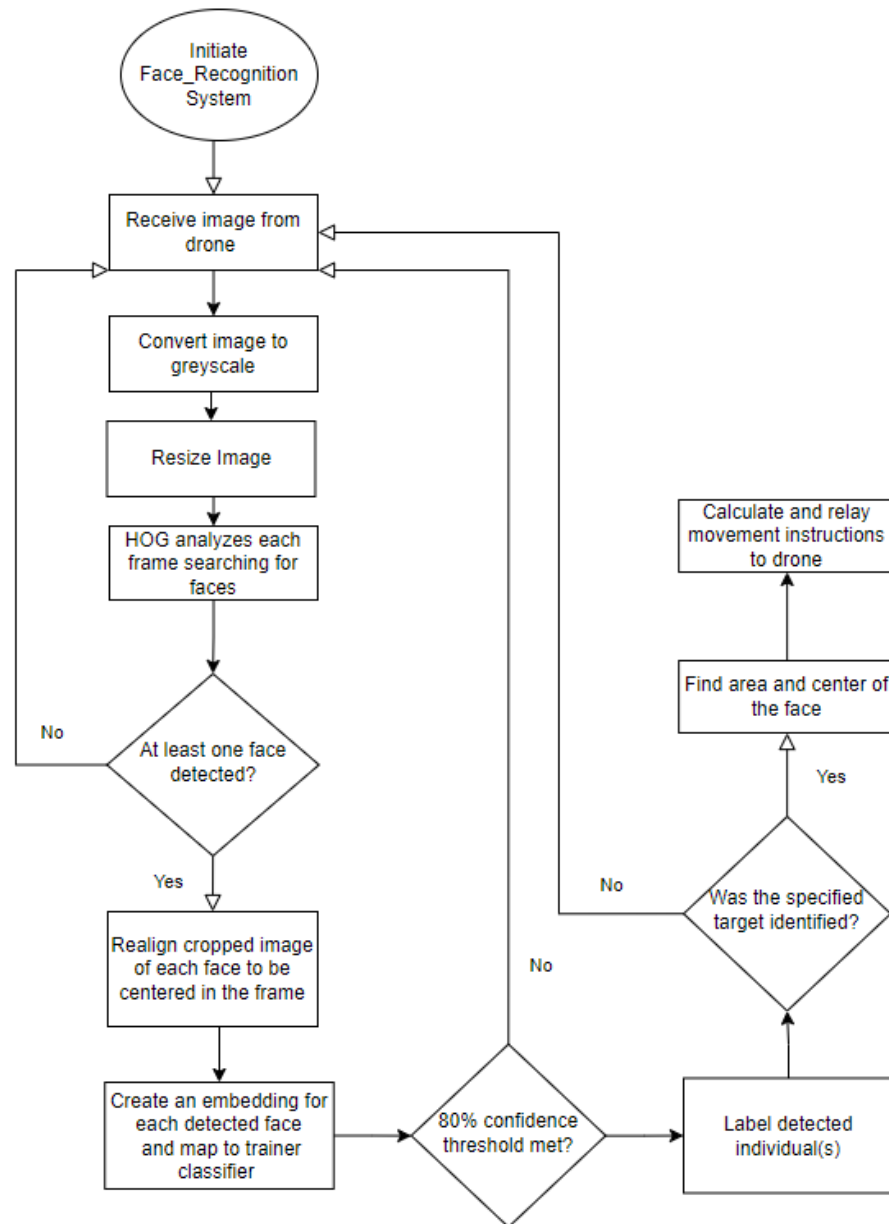


Figure 3.7 Flowchart of the Face_Recognition facial detection and recognition systems.

4. Experiments

The entire system proposed in this thesis was constructed and tested using three different facial recognition systems. Three different facial recognition systems were implemented because they are the key for the entire system to function properly. If the facial detection and recognition algorithms do not perform at a high enough level, then the tracking aspect of the system will not be able to function as designed. Experiments were conducted to collect data points to measure both the accuracy and dependability of the three facial recognition systems that were implemented. Three iterations of each test were conducted and analyzed with the average value being used as the result. All tests were performed in an area with ample lighting. The experiments conducted all use the Ryze Tech Tello drone in conjunction with a computer. The computer used, contains an AMD Ryzen 7 3700X Central Processing Unit (CPU) and a Nvidia GEFORCE GTX 1060 OC GPU. The Intel Wireless AC-9260 Wi-Fi adapter was used to make the connection from the computer to the drone's Wi-Fi network, establishing the link required in order to transmit video and instructions.

4.1 Experiment Design

The first test was used to measure the FPS that each system could analyze. FPS can be used to measure the performance of a system, the more FPS is able to analyze then the more optimized the system is. On the other hand, a system that is analyzing a high amount of FPS might have low accuracy as it isn't doing enough analysis on each frame. This measure is extremely important to our system because there is a target to track. If the system runs too slowly then the target could already be long gone while the system is still analyzing old frames. This test

was done by taking the time when a new frame came into the system and again taking the time when the system was done analyzing that frame. The start time is then subtracted from the end time giving us how long it took to process the frame. By repeating this process for each frame, we can then figure out how many frames are being analyzed per second. This test was done with zero to three people in the frame to see what kind of affect multiple people would have on the system's performance. Each system examined 1000 frames in each iteration of this test. The tests were conducted from a distance of 1 meter and all participants in the frame we're looking directly at the drone's camera to make facial detection and recognition easier for the system. This was done because the efficiency of the program was being measured rather than the accuracy.

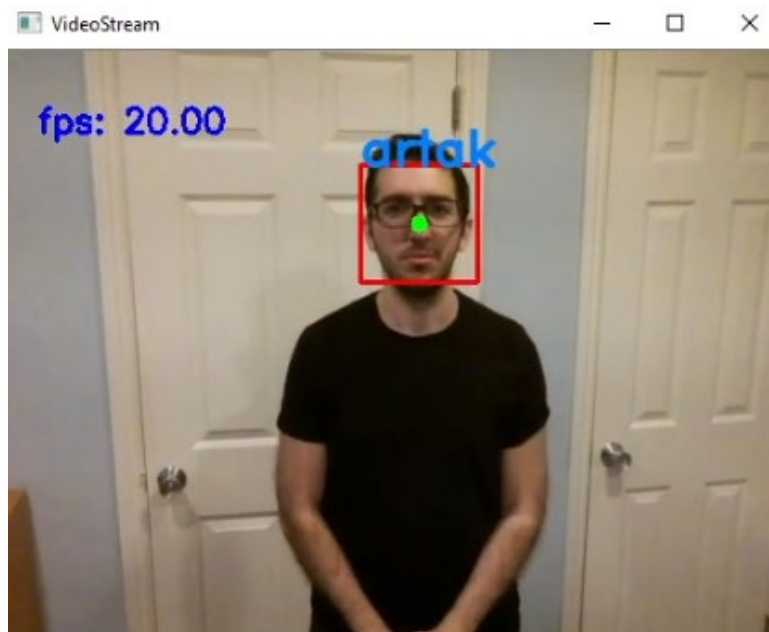


Figure 4.1 LBPH System tracking an individual with FPS display in the top left corner.

The second test was designed to measure the accuracy of all three systems when it comes to both facial detection and facial recognition. An exceedingly accurate facial detection and recognition system is needed for our tracking to work, that is why for all three systems the

minimum confidence threshold for recognizing an individual was set to 80%. This test consisted of multiple variables including the number of people in the frame and the angle at which each person was standing. Each test always consisted of one to three people in the frame. Those people were either directly facing the drone, were looking slightly away from the drone, at a 45 degree angle, or were facing perpendicular from the drone, at a 90 degree angle.

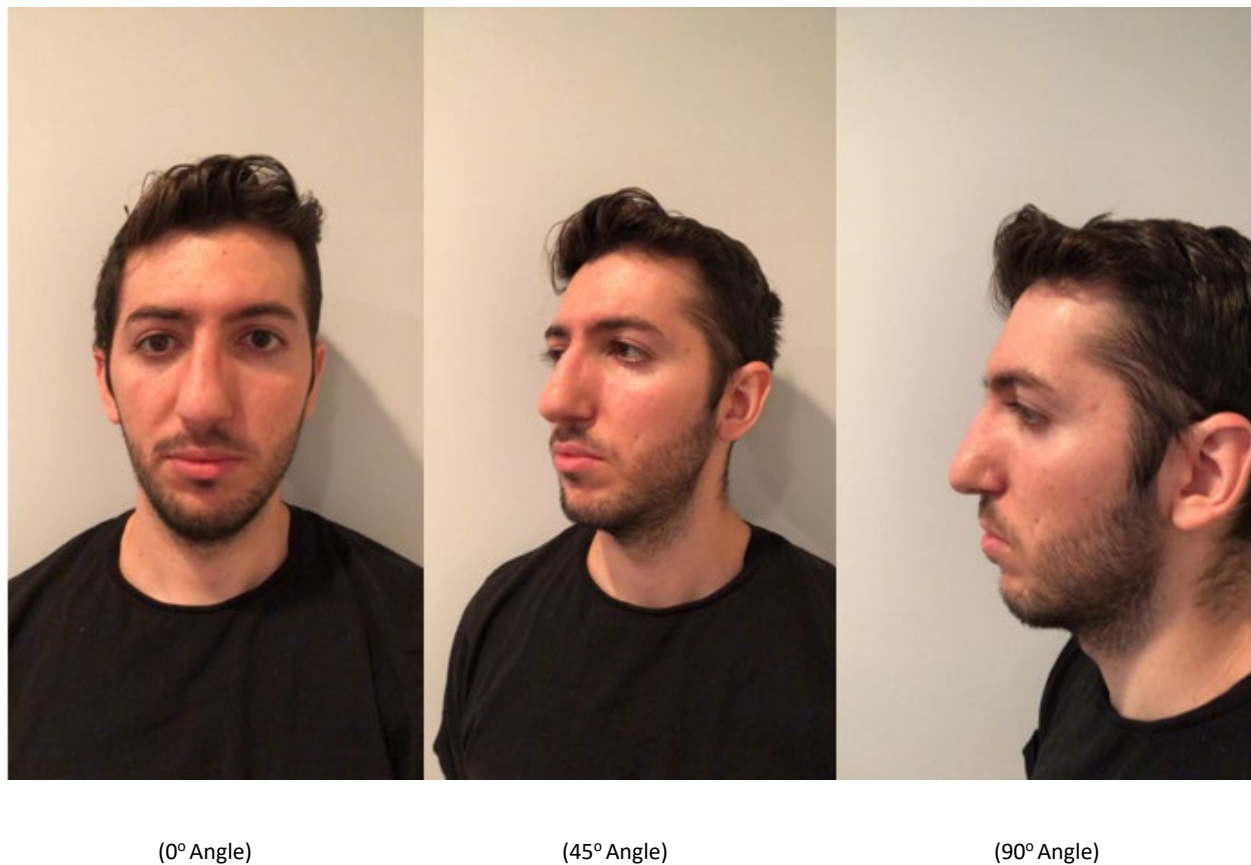


Figure 4.2 Demonstration of different angles the drone was expected to detect faces at.

All the individuals in the frame were facing the same direction in each test, whether it was looking directly in the drone's direction or facing perpendicular from the drone. Experiments were not conducted with individuals facing more than 90 degrees away from the drone because during preliminary testing it was found to be incredibly hard to detect any faces from

that angle and we are testing facial detection and recognition, not head detection and recognition, as from that angle it becomes hard to see a person's face. For this test 100 iterations of the facial detection and recognition process of each system were ran.

The next test was designed to measure the accuracy of the facial detection and recognition systems from a distance. Another tremendously important statistic because it is likely that the drone will be far from the target while searching for them and will need to track the target from a far distance if they are walking away from the drone. In this test, the drone hovered at 2 meters in the air while individuals stood at 1, 1.5, 2, and 2.5 meters from the drone. The test was always conducted with one to three individuals in the frame and all of them were facing the drone. The individuals would all start at a distance of 2.5 meters from the drone and slowly walk towards it stopping every .5 meters to collect data on the system's performance. The main facial detection and recognition loops of each system were ran for 100 iterations every .5 meters.

The next test was also designed to find the accuracy of each facial recognition system. However, it was centered around accuracy in the presence of partial facial obstruction. The purpose of this test is to see how common accessories that result in facial obstruction may affect the systems. With the goal of being able to track a moving target, partial facial obstruction is almost guaranteed, making this test a necessity. Only a single individual was used in these experiments, and they stood about 1 meter away from the drone. For each iteration of testing different parts of the face were partially or fully obstruction and each system's accuracy was measured. The individual completed the test while wearing glasses, sunglasses, a mask,

and partially covering their face vertically, starting at the ear and stopping at the nose. Each system had their main facial detection and recognition loops run for 100 iterations for this test.



(a) Glasses

(b) Sunglasses

(c) Mask

(d) Vertical Obstruction

Figure 4.3 Displaying the different facial obstructions used in testing the facial detection and recognition systems.

The final test was designed to measure the capabilities of each facial recognition system when it came to finding and tracking a target individual in real time. This test was designed to see which facial recognition system functioned best when the entire system was used in a real world setting. The drone was placed in a room with a preprogrammed surveillance path. Tests were conducted with two to five individuals in the room, one of which was the target individual. The target was specified in the program beforehand, and the drone had the job of finding and tracking the target. This test was designed to see if each facial recognition system could pick out the specified target from a group of people and if it could track the target once that individual was found. The drone's starting position was in the center of the room and its

starting height from the ground was 2 meters. All the individuals started the test close to one another, standing in a group.

5. Results

5.1 System Performance

The FPS test was used to determine the runtime efficiency of each system. With no people in the frame the LBPH system averaged 31.22 frames examined every second, FaceNet averaged 41.8 FPS, and Face_Recognition averaged 57.65 FPS. This means with nobody present in the frame, Face_Recognition is the fastest performing system. This would indicate that its facial detection algorithm is faster than the other two, being able to quickly tell that there are no individuals in the frame. However, the story changes as more people become present in the frame. With one person present in the frame LBPH takes about an 8 FPS dip resulting in 23.71 FPS, FaceNet on the other hand took a much larger hit losing about half of its examining proficiency and dropping to 23.66 FPS, Face_Recognition had the largest drop in performance going down to 14.19 FPS. These results seem to indicate that the facial detection algorithms of FaceNet and Face_Recognition have a much better run time and optimization than their facial recognition algorithms. Also, avoiding the computationally heavy process of facial recognition results in a faster system, an expected result as with no one in the frame the systems are not running to their full potential. With each new additional person added LBPH sees an average performance drop of 13%, FaceNet sees a drop of 20%, and Face_Recognition continues its trend of sharp drops at 24%. Face_Recognition's performance drops remarkably fast resulting in only about 1.35 FPS being examined with three individuals in the frame. Overall, Face_Recognition has the largest drops in performance, a drop is expected as more faces are included in a frame since the recognition algorithm will have to analyze each face; however, a system running too slowly would cause issues with the drone's tracking algorithm. With two or

more people in the frame Face_Recognition was running so slow that the frames being examined from the drone were seconds behind an individual's actual location. FaceNet was still able to keep up with its analyzing and LBPH had the lowest overall drop in performance. However, the average decrease in performance shows that given enough individuals in the frame each system will eventually run too slowly for any meaningful real time analysis to take place, which could be used for tracking purposes. Performance can be improved by implementing optimizations like skipping examination for every other frame or by improving the hardware, for example using a more powerful GPU for facial recognition.

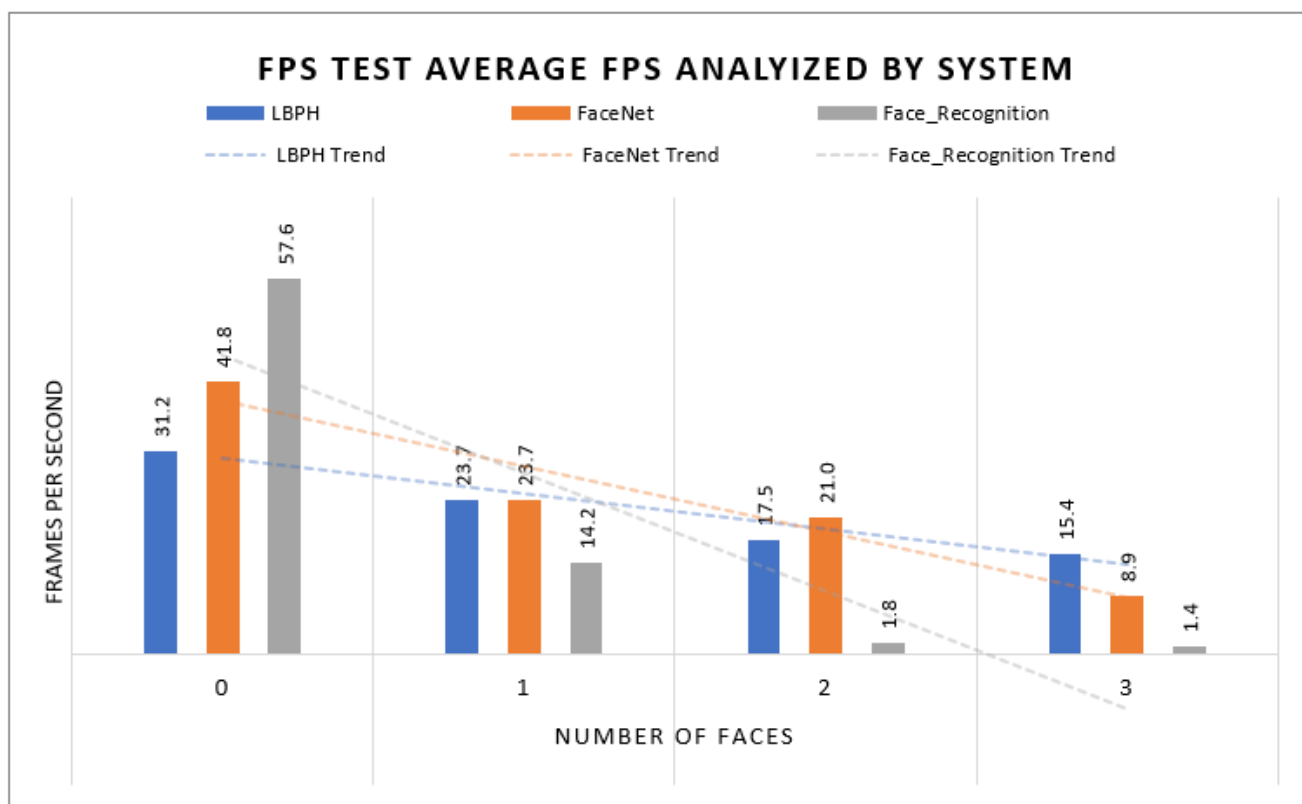


Figure 5.1 Displays the average FPS processed by each system with relation to the number of individuals in each frame.

5.2 Facial Recognition at Angles

Individuals will not always be facing the drone's camera making measuring facial recognition accuracy at angles an important statistic for each system. With only a single person in the frame facing the camera directly LBPH was only able to detect a face 58% of the time, but it was able to accurately identify the detected face 93% of the time. However, both the facial detection and recognition performance drop drastically as the individual turns away from the camera. At a 90 degree angle only 2% of frames with faces in them are detected and of those frames the person is accurately identified only 28% of the time. This gives LBPH the worst performance out of the three systems with one individual in the frame. For the single person test FaceNet performed better, averaging a very high facial detection rate, 98%, and a very high accuracy, 97%, when the individual was directly facing the camera. At a 90 degree angle, the average drops a lot with FaceNet only being able to accurately identify 54% of frames with faces in them and correctly recognizing the individual 53% of the time in those frames. Face_Recognition had the best results with the individual facing the camera directly, detecting a face in the frame 100% of the time and correctly identifying the individual 94% of the time, on average. However, at 90 degrees the performance drastically decreased, only being able to detect a face 12% of the time and correctly identifying that face 49% of the time. FaceNet and Face_Recognition showed much better results and LBPH when testing a single person at multiple angles. This was probably due to the fact that both FaceNet and Face_Recognition realign the faces detected in an image while LBPH does not. This might be giving the two systems the boost of extra accuracy.

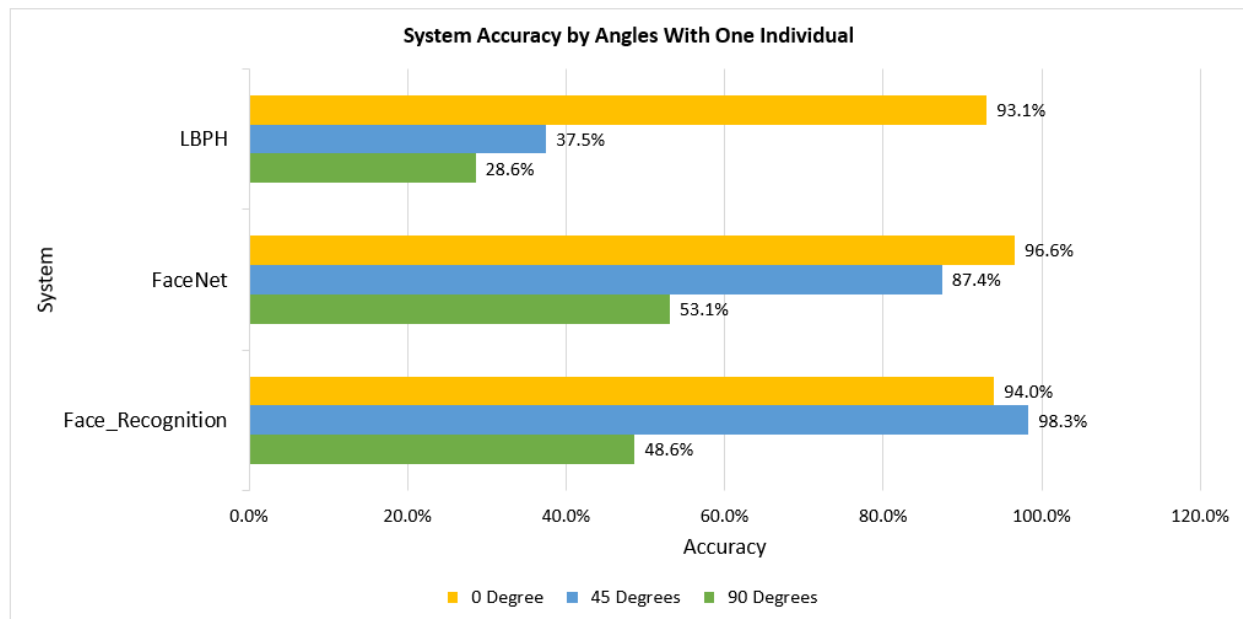


Figure 5.2 Shows the average accuracy of each system when performing facial recognition at an angle. These results include only a single individual in the frame.

For the next portion of this test two individuals were put into frame and tested at all three angles. This test is when false positives started to pop up; luckily, due to the high threshold set for facial recognition, there weren't too many cases. LBPH was able to detect faces in 80% of frames and correctly identify the faces in them 59% of the time, with the individuals looking directly at the camera. With the individuals at a 90 degree angle facial detection dropped to working on only 6% of frames and the accurately of the facial recognition algorithm dropped to 18%. FaceNet continued its trend of high performance by detecting 98% of faces and accurately identifying them 94% of the time. At 90 degrees its performance drops drastically, it still detected 64% of faces but the accuracy dropped to 59% due to a number of misidentified individuals. Face_Recognition kept up its amazing performance with individuals looking directly at the camera by detecting 100% of the frames with faces in them and accurately identifying those people 99% of the time. With the individuals at 90 degrees the facial detection rate

dropped to 55% and the accuracy of the facial recognition dropped to 36%. The increase in facial detection with two individuals in the frame is due to false negatives, where only one out of the two individuals in the frame were correctly detected, this trend continues as more individuals are added to the frame. Face_Recognition was the only system that didn't report any false positives in this test.

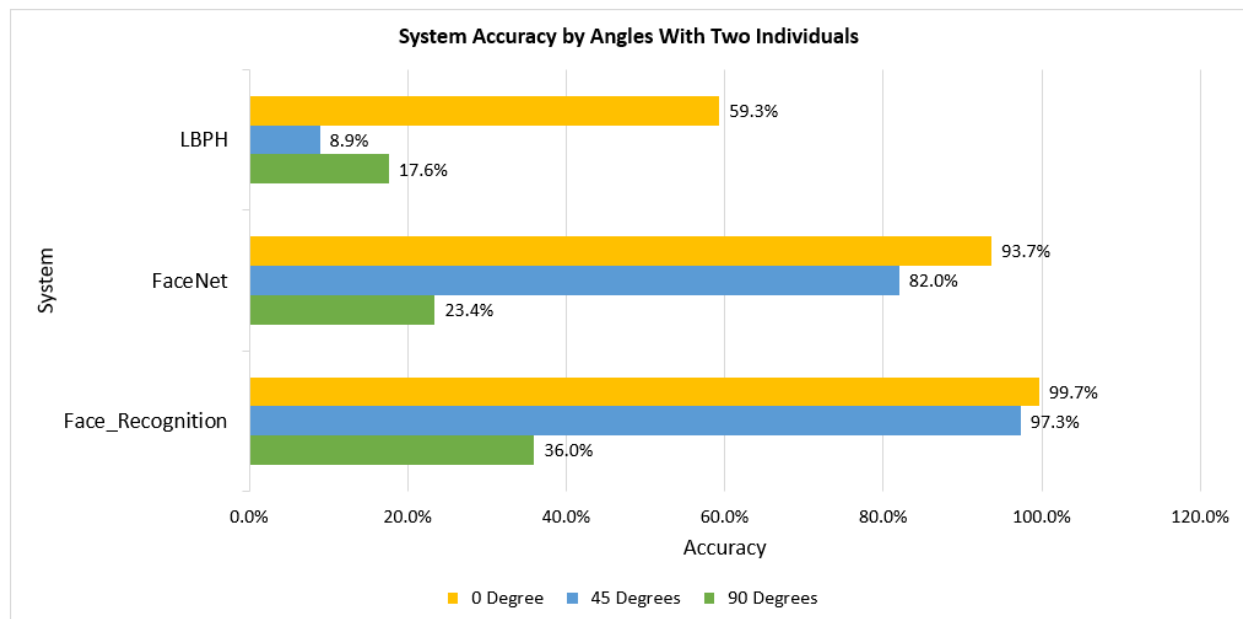


Figure 5.3 Displays the average accuracy of each facial recognition system with two individuals in the frame.

Finally, three individuals were introduced into the frame of the drone's camera. With the individuals facing the camera directly LBPH's accuracy dropped yet again, with the number of false positives increasing, leaving it with a facial recognition accuracy of 43%. The facial detection rate went up to 81%. At 90 degrees the facial detection rate dropped to 12% even with three people in the frame and the accuracy dropped to an even worse 11%. FaceNet fared much better with individuals looking directly at the camera resulting in a 99% detection rate and 85% recognition accuracy. However, at a 90 degree angle it wasn't much better. Facial

detection dropped to 23% and the accuracy of detected individuals dropped to 16%.

Face_Recognition seems to be performing extremely well even while individuals are added to the frame. At a 0 degree angle it had a 100% facial detection rate and a 99% accuracy. At 90 degrees it had a 71% facial detection rate and 41% accuracy. Again, the facial detection rates went up as more individuals are added to the frame but not all individuals were detected in all of those frames.

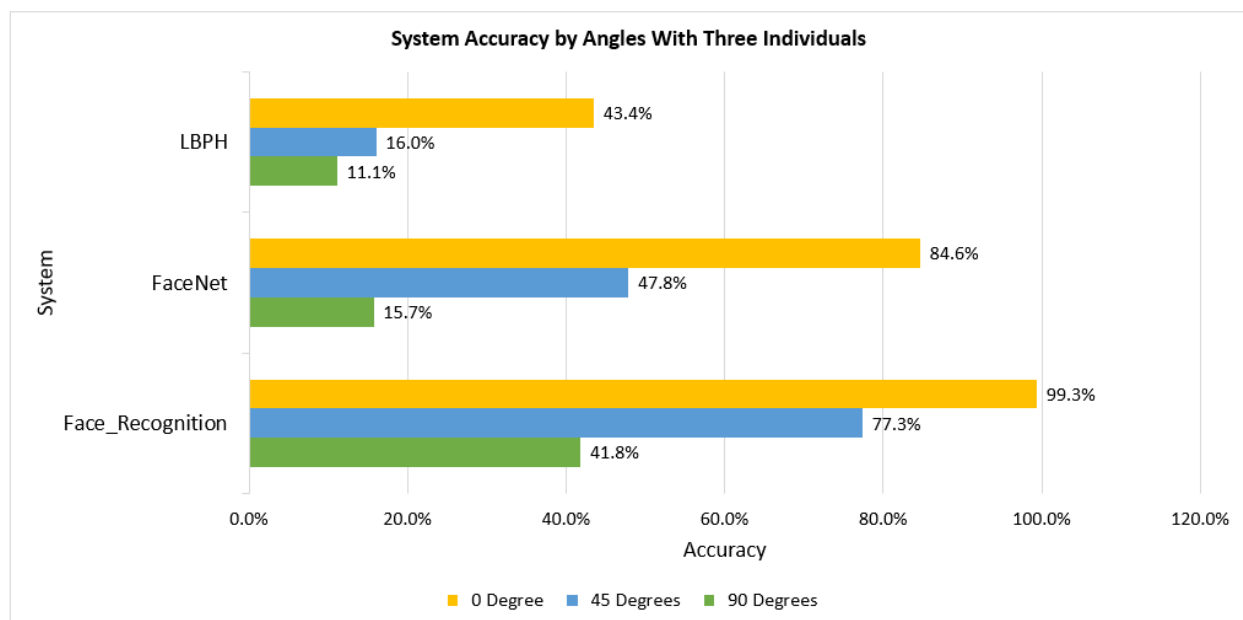


Figure 5.4 Depicts the average accuracy of each facial recognition system when three individuals are in the frame.

FaceNet and Face_Recognition seemed to be a cut above LBPH for the purposes of these tests.

Both systems had consistently higher facial detection and recognition rates including having fewer numbers of false positives and false negatives. The fact that both of these systems realign images may be the cause of the increased accuracy. Particularly the face landmarking algorithm present in Face_Recognition seems to be making the difference with the system's improved

accuracy. Overall, Face_Recognition seems to be the best system to use to identify individuals at an angle due to its ability to limit the number of false positives.

5.3 Facial Recognition from A Distance

While searching for an individual, their distance from the drone will have a large impact on the accuracy of the facial detection and recognition algorithms. If an individual is too far, they will appear in a small area in the streamed frame. If that face is detected the recognition systems could have trouble processing such a small and unclear image. All three systems resize any faces detected and crop them for recognition to help alleviate this issue but nonetheless it can have a huge effect on accuracy as the results indicate.

At 1 meter the accuracy of all three systems was very similar to that of the angle test with individuals looking straight at the drone's camera. The angle test was also conducted at 1 meter so the similarity in accuracy is expected. FaceNet and Face_Recognition showed amazing accuracy regardless of the number of people in the frame. LBPH had great accuracy with a single person in the frame but due to its high number of false positives its accuracy quickly dropped as more people were added to the frame.

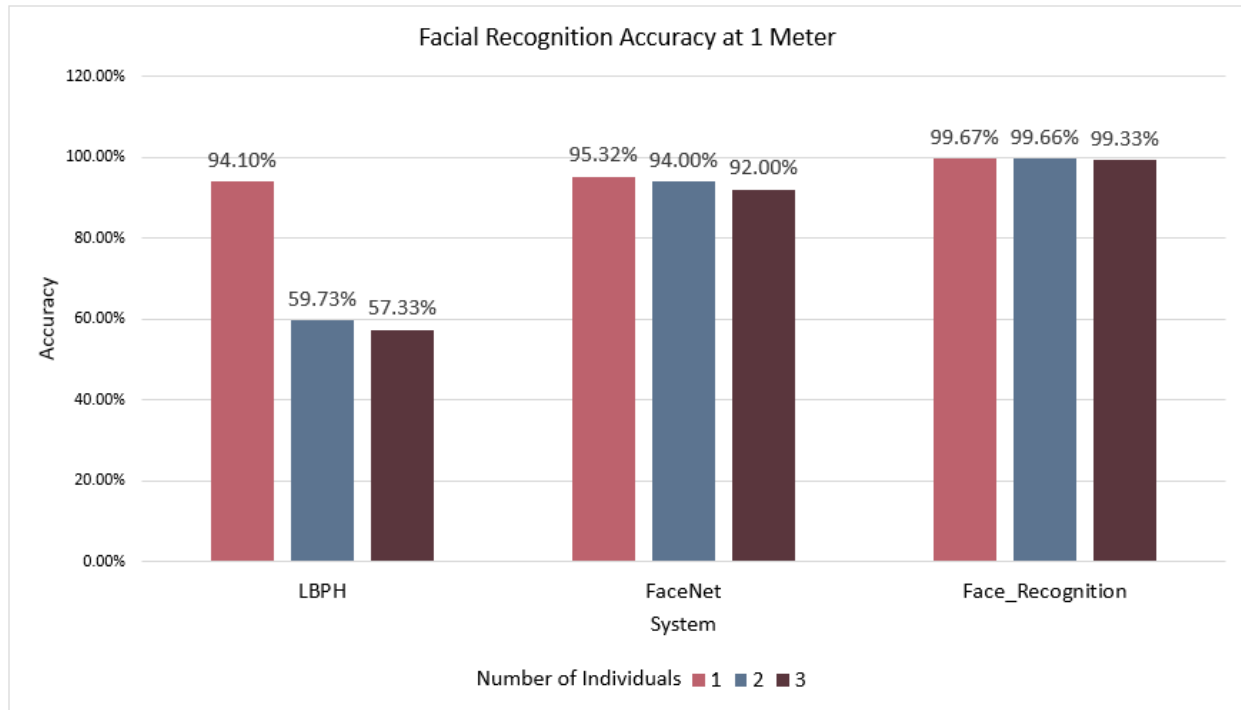


Figure 5.5 Accuracy of each system at a 1 meter distance from individuals.

At 1.5 meters LBPH had the strangest results, as distance increased the expected outcome would be that the accuracy decreases. However, the accuracy for LBPH slightly increased, while more than 1 individual was in frame, going from 59% to 66% accuracy with two people in the frame and from 57% to 62% accuracy with three people in the frame. With 1 to 3 people in the frame FaceNet and Face_Recognition both had very high accuracy rates at this distance. FaceNet averaged an 81% accuracy and Face_Recognition averaged 98% accuracy with one to three people in the frame.

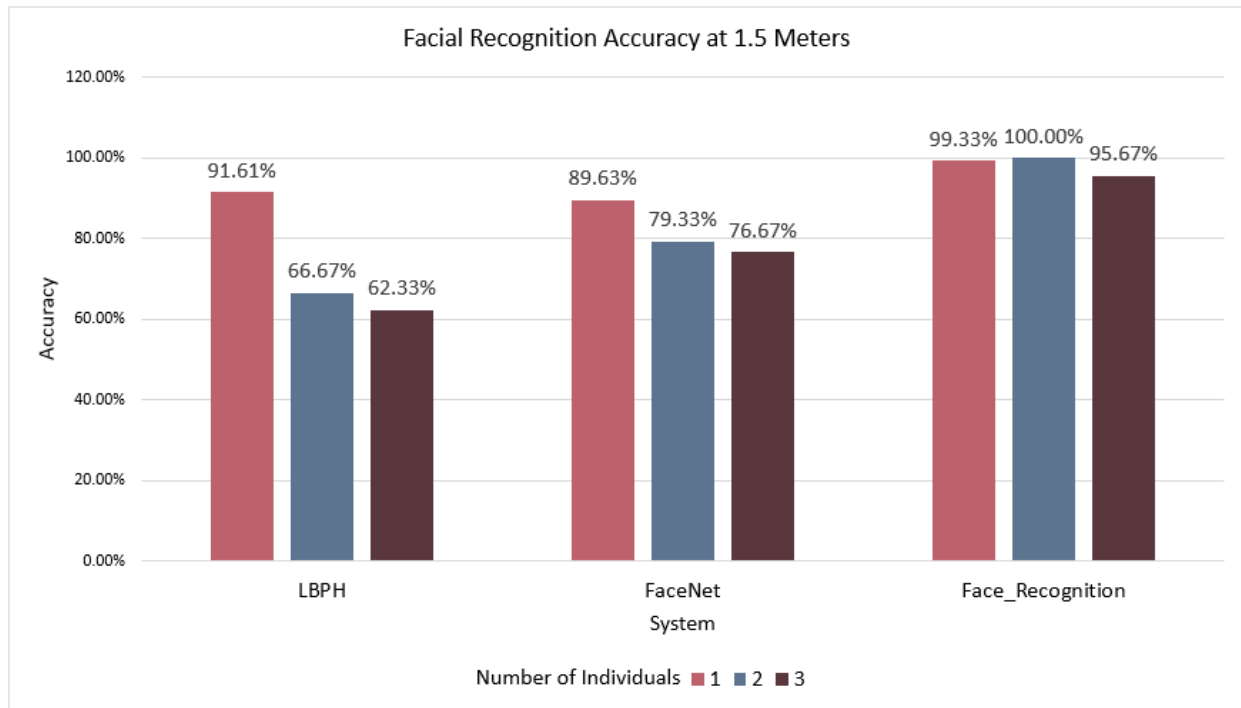


Figure 5.6 Accuracy of each system at a 1.5 meter distance from individuals.

With individuals at 2 meters LBPH continued its trend of increased accuracy, seeing its largest gains yet, with its average accuracy increasing from 73%, from the 1.5 meter test, to 83%. Its largest gain occurred in the test with two individuals with accuracy increasing by 16%. FaceNet experiences a drop in accuracy, which is the expected result as distance increases. The system is still able to detect faces remarkably well but the facial recognition algorithm was struggling to keep up. The average accuracy for one to three people tests dropped to 67%, when it was 81% in the previous test. Face_Recognition had the most alarming results as its facial detection algorithm failed to detect any faces at this distance. Even with three individuals in the frame not a single face was detected. How the facial recognition portion of Face_Recognition would have fared at this distance is unknown because with no detected faces that portion of the algorithm cannot run.

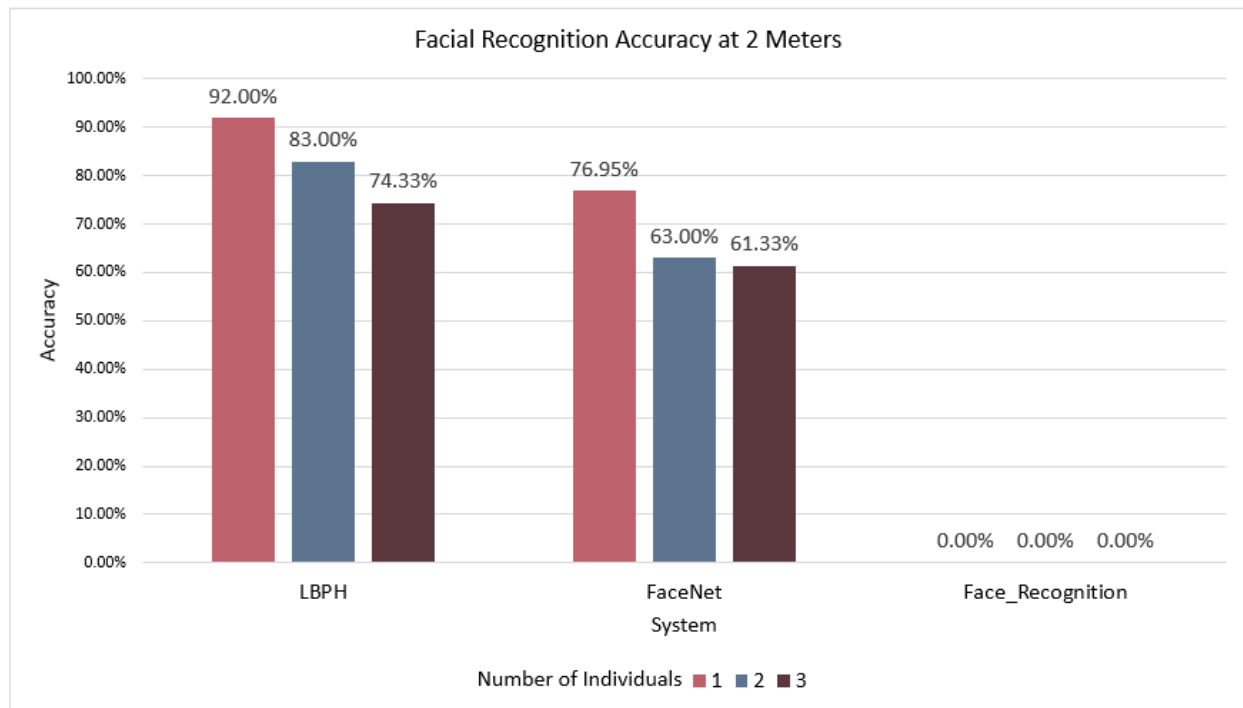


Figure 5.7 Accuracy of each system at a 2 meter distance from individuals.

Finally, when individuals were at 2.5 meters LBPH started to show a drastic dip in accuracy. The average accuracy for the previous distance was 83% but at 2.5 meters it fell to 46%. This dip in accuracy was due to the extremely high number of false positives. FaceNet also experienced a huge decline in accuracy, averaging an accuracy of 22% for one to three people in the frame. FaceNet's accuracy dipped on average 17% for every .5 meters the individuals moved away from the drone. Face_Recognition continued its trend from the 2 meter test and was unable to detect any faces in the provided frames.

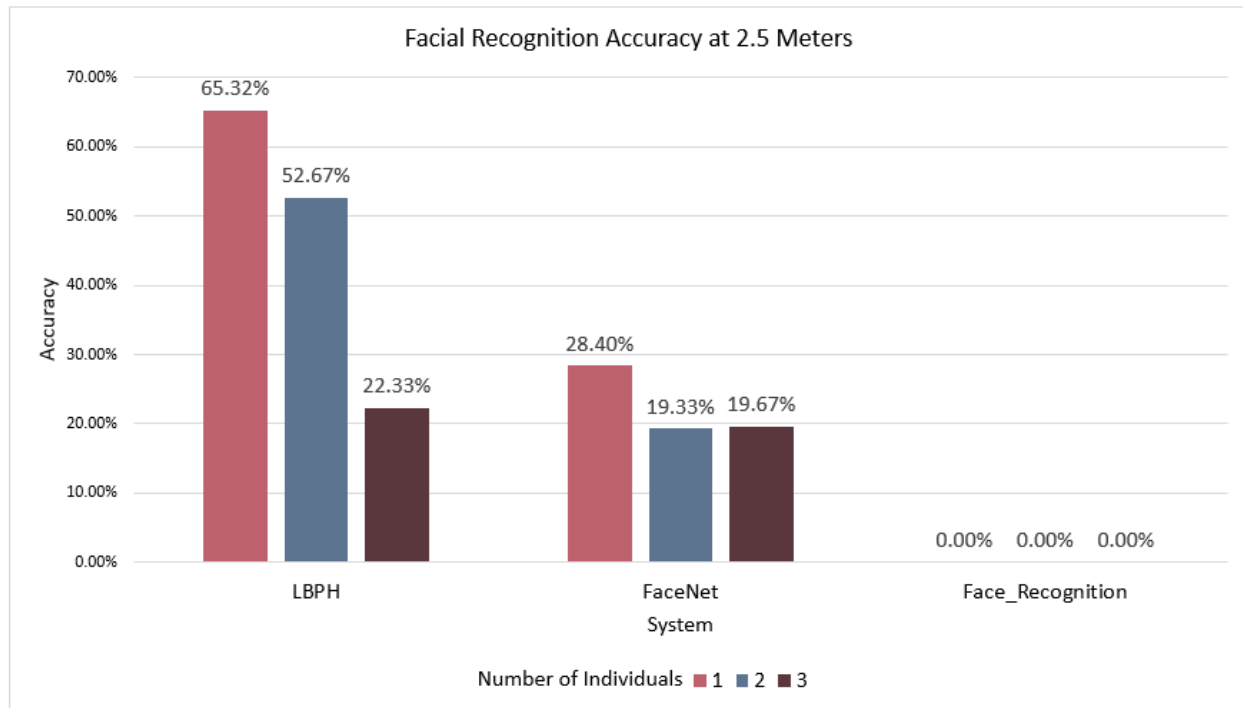


Figure 5.8 Accuracy of each system at a 2.5 meter distance from individuals.

Overall, distance seems to be the largest obstacle so far when it comes to accuracy. At a relatively short distance of 2.5 meters two of the algorithms had terrible accuracy and the third one didn't function at all. FaceNet experienced multiple sharp declines in accuracy meaning that it might not be the best for long distance facial recognition. LBPH had the best overall accuracy, but its large number of false positives is cause for concern. Face_Recognition had showed consistently the best accuracy in the previous experiment but in this one its weakness was exhibited. Some of these distance issues might be alleviated with the use of a drone that has a more capable camera streaming images in a higher resolution.

5.4 Facial Obstruction

Facial obstruction can pose a huge threat to facial recognition systems. Common accessories worn by people every day may make facial recognition systems lose a chunk of their accuracy or

make the system entirely void. In the three systems tested, facial obstruction had a huge effect on the accuracy of the results, the zero degree with a single individual in frame results from the angle test will be used as the baseline to compare the results from this test. Glasses seemed to have the lowest effect on accuracy, LBPH was still 91% accurate, FaceNet was 84% accurate, and Face_Recognition was 100% accurate, in terms of facial recognition. Even though glasses are a very common accessory, they don't cover too much of the face and resulted in very minimal or no loss to accuracy for all three systems. Sunglasses had a larger effect, with LBPH showing the largest difference in accuracy, dropping all the way down to 76%. FaceNet and Face_Recognition also experienced dips but they were extremely minute. One of the reasons for the large dip in LBPH maybe due to the histogram it uses for facial recognition. The facial detection part of the system still worked well, detecting a face in the frame 92% of the time.

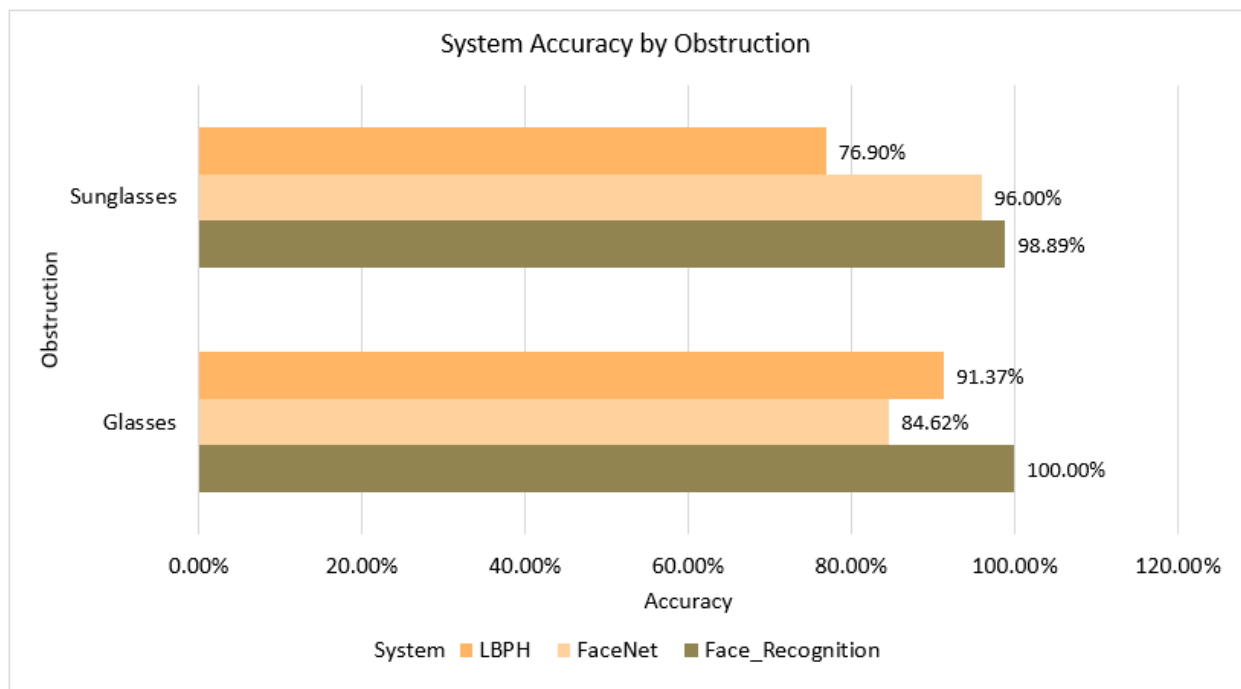


Figure 5.9 Accuracy of each system when an individual is wearing glasses or sunglasses.

Masks have seen an exponential growth in use and their use might be around for a long time to come. Masks cover a large portion of the face, from the chin all the way to the nose, and gave all three systems their first taste of some real trouble. LBPH still had a high level of accuracy at 89%; however, its facial detection system performed poorly, detecting a face in only 48% of the frames. FaceNet produced the most shocking results, with its accuracy dropping to 0%. The system didn't result in any false positives it just couldn't recognize anyone within the 80% threshold. The MTCNN used by FaceNet was only able to detect faces in 13% of the frames it was shown, and no faces were recognized in any of those frames. Even though the exact representation of the 128 measurements taken by FaceNet are unknown, it should be safe to say that a mask blocks some of the key areas checked by the system. In general, Face_Recognition performed the best out of the three systems in this test. It was able to detect a face in 68% of frames and in those detected frames it accurately recognized the individual 85% of the time. Face_Recognition's use of HOG for facial detection may account for its better facial detection performance when compared to the other systems. Face_Recognition also uses a 128-measurement embedding for facial recognition, just like FaceNet, but it could be measuring different parts of the face, resulting in the huge difference between the two system's accuracies.

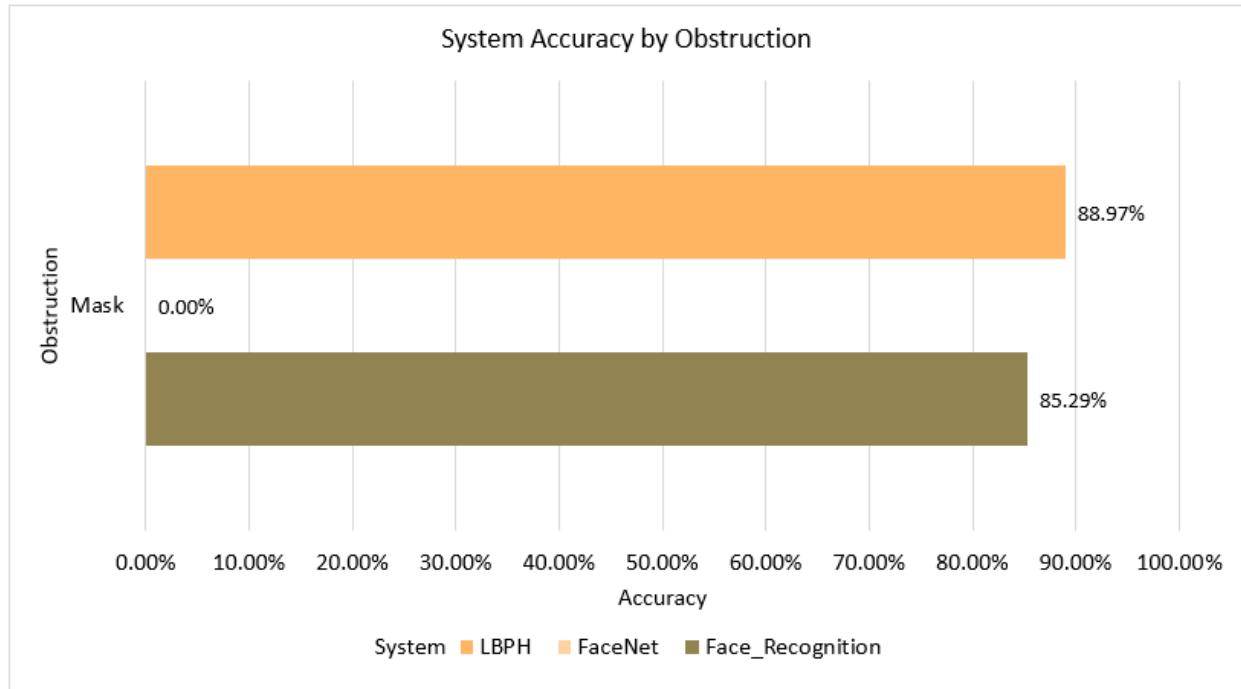


Figure 5.10 Accuracy of each system when an individual is wearing a mask.

So far, the obstructions have been mainly horizontal coverings of the face. For the final test, the face was covered vertically as shown in Figure 4.3. This test seemingly caused more issues for the facial detection algorithms of each system than it did for the facial recognition portion. LBPH displayed its highest accuracy yet, correctly recognizing 100% of the faces detected; however, only 2% of the frames with faces in them were detected, its poorest performance yet. Only in the third iteration of this test did LBPH detect or recognize any faces, failing to detect or recognize even a single face in the other iterations of this test. Leading to the conclusion that this test was the exception and that LBPH generally will have terrible accuracy with this type of facial obstruction. FaceNet did not fare any better only detecting faces in .6% of frames and not being able to recognize any, again the 128 measurements used by FaceNet might have measurements of the side of the face being covered which results in no match being produced.

Face_Recognition was able to detect faces in 6% of the frames and accurately identified the individual 100% of the time. In more tests than not, it was able to detect a person and accurately identify them, so unlike LBPH, it seems like Face_Recognition may actually be able to recognize faces with this type of obstruction. This also reinforces the idea that Face_Recognition and FaceNet have vastly different measurements used for their embeddings. Overall, it appears the Face_Recognition maybe the best system to use when accounting for facial obstructions.

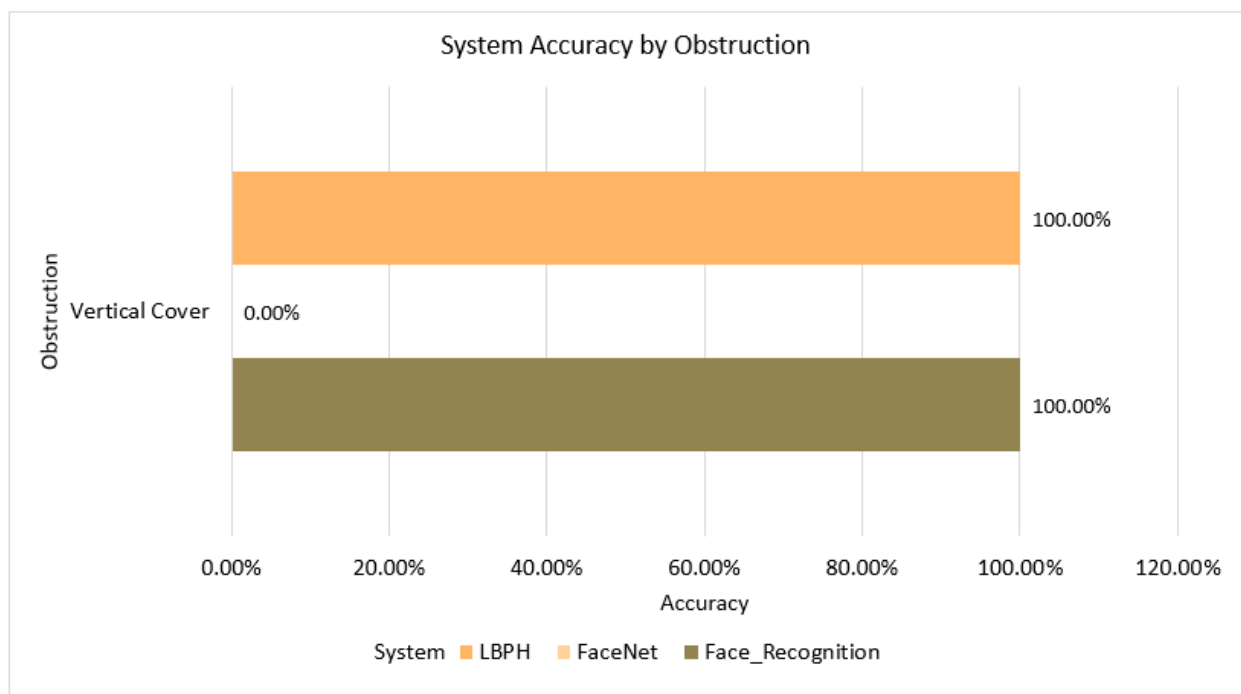


Figure 5.11 Accuracy of each system when an individual has their face covered vertically.

5.5 Target Recognition and Tracking

This test was designed to see how the implemented system functioned as a whole, which each facial recognition system. LBPH started this test with promising results. With two individuals in the group, it was able to correctly identify and track the target in all five tests. With three individuals, it was also able

to locate the target and track them in all five tests as well; however, the LBPH system misidentified individuals in some frames. This resulted in incorrect movement instructions being sent to the drone. With only three individuals in the group, it was not too much of an issue because the target was correctly identified in most frames. Also, once tracking began and the target moved further away from the group, the system worked as it should. With four individuals in the group, performance started to deteriorate. The number of false positives rose, causing the system to issue numerous incorrect movement instructions to the drone. Only in two of the five tests with four individuals in the group was the drone able to identify and track the target. This trend continued as a fifth individual was added to the group. Only during one test, out of the five, was the drone able to correctly identify and track the target. The large number of incorrect movement commands, caused by the false positives, being issued to the drone almost made the tracking system obsolete as the drone started to move around aimlessly. The results showed that with a large group of individuals, LBPH could not provide the accuracy necessary for the tracking system to work.

FaceNet was the next system tested. With two individuals in the group, it was able to find and track the specified target in all five tests. It had the same results for three and four individuals in the group as well, picking out and tracking the specified target in all those tests. The false positive rate was minimal, so it didn't have the same issue of numerous incorrect movement commands being sent, like the LBPH system had. FaceNet started to struggle with five individuals in the group, only being able to correctly identify and track the target individual in three out of the five tests. The issue wasn't the accuracy of the system, it was its performance. With five faces in the frame at once, FaceNet was struggling to perform real time analysis. In the two tests that the system failed, analysis of the frames were seconds behind the images streamed by the drone. This caused the system to issue movement commands that were tracking the location of where the individuals use to be and not where they were currently located. This

delay resulted in the drone losing track of the target individual. FaceNet showed better results than LBPH, but again issues arose once too many people were in the frame.

Face_Recognition started to show problems with only two individuals in the frame, finding and tracking the target in only 1 out of five attempts. The system's performance only got worse with the addition of a third individual. The system performed extremely poorly, not being able to find and track an individual in any of the tests. This was due to the poor runtime performance of Face_Recognition, as it was only able to examine about one frame a second. As more individuals were added to the group, it only got worse as Face_Recognition ran at a greater delay, causing the system to issue movement commands seconds behind the target individual's actual location. Face_Recognition made tracking impossible as the system's delay removed the ability to perform any meaningful real time analysis.

6. Conclusion

In this thesis a system was proposed and implemented to perform real time facial detection and recognition using a drone, in conjunction with a computer, to find and track a specified target individual. The facial recognition systems were found to be the key component of the entire system, so three different facial recognition systems were implemented to quantify their performance. All though, the different facial recognition systems had varying strengths and weaknesses, FaceNet performed best when the system was tested in a real-world setting. The results of this thesis indicate that using facial recognition algorithms in conjunction with a drone with the goal of tracking an individual is possible. Even when using low cost, light weight drones, facial recognition is possible by having the drone stream video to a computer powerful enough to handle facial recognition. By streaming video to a computer, this system solved many of the performance issues mentioned in papers that implemented similar systems [2;7]. There exist extraneous factors which limit the capabilities of this system; but it served as a strong foundation that can be built upon.

References

- [1] Amato, Giuseppe, et al. "Multi-Resolution Face Recognition with Drones." 2020 3rd International Conference on Sensors, Signal and Image Processing, 2020, <https://doi.org/10.1145/3441233.3441237>.
- [2] Deeb, Adam, et al. "Drone-Based Face Recognition Using Deep Learning." Advances in Intelligent Systems and Computing, 2020, pp. 197–206., https://doi.org/10.1007/978-981-15-3383-9_18.
- [3] Geitgey, Adam. "Machine Learning Is Fun! Part 4: Modern Face Recognition with Deep Learning." Medium, Medium, 24 Sept. 2020, <https://medium.com/@ageitgey/machine-learning-is-fun-part-4-modern-face-recognition-with-deep-learning-c3cffc121d78>.
- [4] Han, Song, et al. "Deep Drone: Object Detection and Tracking for Smart Drones on Embedded System." Stanford.edu, 2016, https://web.stanford.edu/class/cs231a/prev_projects_2016/deep-drone-object_2.pdf.
- [5] Hsu, Hwai-Jung, and Kuan-Ta Chen. "Face Recognition on Drones: Issues and Limitations." Proceedings of the First Workshop on Micro Aerial Vehicle Networks, Systems, and Applications for Civilian Use, 2015, <https://doi.org/10.1145/2750675.2750679>.
- [6] K.G., Shanthi, et al. "Smart Drone with Real Time Face Recognition." Materials Today: Proceedings, 2021, <https://doi.org/10.1016/j.matpr.2021.07.214>.
- [7] Pareek, Bhavya, et al. "Person Identification Using Autonomous Drone through Resource Constraint Devices." 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS), 2019, <https://doi.org/10.1109/iotsms48152.2019.8939254>.
- [8] Prado, Kelvin Salton do. "Face Recognition: Understanding LBPH Algorithm." Medium, Towards Data Science, 3 Feb. 2018, <https://towardsdatascience.com/face-recognition-how-lbph-works-90ec258c3d6b>.
- [9] Schroff, Florian, et al. "FaceNet: A Unified Embedding for Face Recognition and Clustering." 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, <https://doi.org/10.1109/cvpr.2015.7298682>.
- [10] Viola, Paul and Michael Jones, "Rapid object detection using a boosted cascade of simple features," *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*, 2001, <https://doi.org/10.1109/CVPR.2001.990517>.
- [11] Wang, Li, and Ali Akbar Siddique. "Facial Recognition System Using LBPH Face Recognizer for Anti-Theft and Surveillance Application Based on Drone Technology." Measurement and Control, vol. 53, no. 7-8, 2020, pp. 1070–1077., <https://doi.org/10.1177/0020294020932344>.